OOPJ LAB ASSIGNMENT – 10

Name – Nishant Nahar

Roll No – 241551078

1. Write a Java program that creates two threads: one prints even numbers and the other prints odd numbers simultaneously.

```
class EvenThread extends Thread {
   int limit;
   EvenThread(int l){ limit=l; }
   public void run(){
      for(int i=2;i<=limit;i+=2) System.out.print(i+" ");
   }}
class OddThread extends Thread {
   int limit;
   OddThread(int l){ limit=l; }
   public void run(){
      for(int i=1;i<=limit;i+=2) System.out.print(i+" ");
   }}
class as_10_q_1 {
   public static void main(String[] args){
      System.out.println("Nishant - 241551078");
      int n = 10;
      new EvenThread(n).start();
      new OddThread(n).start();
   }}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\li
Nishant - 241551078
1 3 5 7 9 2 4 6 8 10
```

2. Implement multithreading using both Thread class and Runnable interface in the same program and compare their outputs.

```
class MyThread2 extends Thread {
    public void run() {
        System.out.println("Thread class running");
    }}
class MyRunnable2 implements Runnable {
    public void run() {
        System.out.println("Runnable running");
    }}
class as_10_q_2 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        new MyThread2().start();
        new Thread(new MyRunnable2()).start();
    }}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
Thread class running
Runnable running
```

3. Write a program where three threads print messages at different intervals using Thread.sleep().

```
class T1 extends Thread {
    public void run() {
        try {
            System.out.println("T1");
            Thread.sleep(1000);
        } catch (Exception e) {}}}
```

```java
class T2 extends Thread {
    public void run() {
        try {
            System.out.println("T2");
            Thread.sleep(1500);
        } catch (Exception e) {}}}
class as_10_q_3 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        new T1().start();
        new T2().start();
    }}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
T1
T2
```

4. Create a program where five threads print their thread name, ID, and priority while executing.

```java
class InfoThread4 extends Thread {
    public void run() {
        System.out.println("Name: " + getName() + " | ID: " + getId() + " | Priority: " +
getPriority());
    }}
class as_10_q_4 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        for (int i = 0; i < 5; i++) new InfoThread4().start();
    }}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
Name: Thread-4 | ID: 40 | Priority: 5
Name: Thread-1 | ID: 37 | Priority: 5
Name: Thread-3 | ID: 39 | Priority: 5
Name: Thread-2 | ID: 38 | Priority: 5
Name: Thread-0 | ID: 36 | Priority: 5
```

5. Write a program that dynamically creates n threads based on user input and executes a simple task.

```java
class Task5 extends Thread {
    public void run() {
        System.out.println("Running " + getName());
    }}
public class as_10_q_5 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        int n = 5;
        for (int i = 0; i < n; i++) new Task5().start();
    }}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
Running Thread-2
Running Thread-3
Running Thread-1
Running Thread-4
Running Thread-0
```

6. Write a Java program to display the state of a thread at different stages using getState().

```java
class StateThread6 extends Thread {
    public void run() {
        System.out.println("Thread started");
    }}
public class as_10_q_6 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        StateThread6 t = new StateThread6();
        System.out.println("Before start: " + t.getState());
        t.start();
        System.out.println("After start: " + t.getState());
    }}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
Before start: NEW
After start: RUNNABLE
Thread started
```

7. Create a program where one thread monitors the state changes of another thread throughout its lifecycle.

```java
class Worker7 extends Thread {
    public void run() {
        for (int i = 0; i < 3; i++) System.out.println("Working..."); }}
class as_10_q_7 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        Worker7 w = new Worker7();
        System.out.println("Initial: " + w.getState());
        w.start();
        while (w.isAlive()) {
            System.out.println("State: " + w.getState());}
        System.out.println("Final: " + w.getState());}}
```

OUTPUT

```
State: RUNNABLE
State: RUNNABLE
State: RUNNABLE
State: RUNNABLE
State: RUNNABLE
State: RUNNABLE
Working...
Working...
Working...
State: RUNNABLE
Final: TERMINATED
```

8. Implement a program that demonstrates NEW → RUNNABLE → TERMINATED thread states clearly.

```java
class Demo8 extends Thread {
    public void run() {
        System.out.println("Thread executing");}}
public class as_10_q_8 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        Demo8 t = new Demo8();
        System.out.println("State: " + t.getState());
        t.start();
        System.out.println("State after start: " + t.getState());}}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\l
Nishant - 241551078
State: NEW
State after start: RUNNABLE
Thread executing
```

9. Write a program demonstrating thread priority scheduling by assigning different priorities to three threads.

```
class Priority9 extends Thread {
    public void run() {
        System.out.println(getName() + " priority " + getPriority());
    }}
public class as_10_q_9 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        Priority9 t1 = new Priority9(), t2 = new Priority9(), t3 = new Priority9();
        t1.setPriority(Thread.MIN_PRIORITY);
        t2.setPriority(Thread.NORM_PRIORITY);
        t3.setPriority(Thread.MAX_PRIORITY);
        t1.start();t2.start();t3.start();
    }}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
Thread-2 priority 10
Thread-1 priority 5
Thread-0 priority 1
```

10. Implement a simulation where threads represent tasks arriving at different times, showing how scheduling occurs.

```
class Task10 extends Thread {
    int delay;
    Task10(int d) {
        delay = d; }
    public void run() {
        try {
            Thread.sleep(delay);
        } catch (Exception e) {}
        System.out.println("Task " + getName() + " executed");}}
public class as_10_q_10 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        new Task10(1000).start();
        new Task10(2000).start();
        new Task10(3000).start();
    }}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\l
Nishant - 241551078
Task Thread-0 executed
Task Thread-1 executed
Task Thread-2 executed
```

11. Write a program demonstrating the difference between calling run() directly and start().

```
class Demo11 extends Thread {
    public void run() {
        System.out.println("Thread running");}}
class as_10_q_11 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        Demo11 t = new Demo11();
        t.run();t.start();}}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\l
Nishant - 241551078
Thread running
Thread running
```

12. Write a program where multiple threads print numbers but pause execution using sleep().

```
class NumThread12 extends Thread {
    public void run() {
        try {
            for (int i = 1; i <= 5; i++) {
                System.out.println(getName() + " " + i);
                Thread.sleep(500);
            }} catch (Exception e) {}}}
class as_10_q_12 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        new NumThread12().start();
        new NumThread12().start();
    }}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
Thread-1 1
Thread-0 1
Thread-0 2
Thread-1 2
Thread-1 3
Thread-0 3
```

13. Create a program where the main thread waits for child threads using join().

```
class Child13 extends Thread {
    public void run() {
        for (int i = 1; i <= 3; i++) System.out.println("Child " + i); }}
class as_10_q_13 {
    public static void main(String[] args) throws Exception {
        System.out.println("Nishant - 241551078");
        Child13 t = new Child13();
        t.start();t.join();}}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
Child 1
Child 2
Child 3
```

14. Write a program where thread names are changed dynamically using setName() and retrieved using getName().

```
class NameThread14 extends Thread {
    public void run() {
        System.out.println("Thread: " + getName());}}
class as_10_q_14 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        NameThread14 t = new NameThread14();
        t.setName("WorkerThread");
        t.start();}}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
Thread: WorkerThread
```

15. Write a program where multiple threads access a shared counter without synchronization and observe inconsistent results.

```
class Counter15 {
    int count = 0;
```

```
    void increment() {
        count++;
    }}
class Worker15 extends Thread {
    Counter15 c;
    Worker15(Counter15 c) {
        this.c = c;}
    public void run() {
        for (int i = 0; i < 1000; i++) c.increment();}}
public class as_10_q_15 {
    public static void main(String[] args) throws Exception {
        System.out.println("Nishant - 241551078");
        Counter15 c = new Counter15();
        Worker15 t1 = new Worker15(c), t2 = new Worker15(c);
        t1.start();t2.start();t1.join();t2.join();
        System.out.println("Count: " + c.count);
    }}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
Count: 2000
```

16. Modify the above program using a synchronized method to fix the inconsistency.

```
class Counter16 {
    int count = 0;
    synchronized void increment() {
        count++;}}
class Worker16 extends Thread {
    Counter16 c;
    Worker16(Counter16 c) {
        this.c = c;}
    public void run() {
        for (int i = 0; i < 1000; i++) c.increment();}}
public class as_10_q_16 {
    public static void main(String[] args) throws Exception {
        System.out.println("Nishant - 241551078");
        Counter16 c = new Counter16();
        Worker16 t1 = new Worker16(c), t2 = new Worker16(c);
        t1.start();t2.start();t1.join();t2.join();
        System.out.println("Count: " + c.count);
    }}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3
Nishant - 241551078
Count: 2000
```

17. Write a program demonstrating synchronized block instead of a synchronized method.

```
class Counter17 {
    int count = 0;
    void increment() {
        synchronized (this) {
            count++;
        }}}
class Worker17 extends Thread {
    Counter17 c;
    Worker17(Counter17 c) {
        this.c = c;}
    public void run() {
        for (int i = 0; i < 1000; i++) c.increment();}}
class as_10_q_17 {
    public static void main(String[] args) throws Exception {
        System.out.println("Nishant - 241551078");
        Counter17 c = new Counter17();
```

```
        Worker17 t1 = new Worker17(c), t2 = new Worker17(c);
        t1.start();t2.start();t1.join();t2.join();
    }}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
```

18. Create a banking system where multiple threads try to withdraw money simultaneously and use synchronization to prevent incorrect balances.

```
class Bank18 {
    int balance = 1000;
    synchronized void withdraw(int amt) {
        if (balance >= amt) {
            balance -= amt;
            System.out.println("Withdrawn " + amt);
        } else System.out.println("Insufficient");}}
class Customer18 extends Thread {
    Bank18 b;
    Customer18(Bank18 b) {
        this.b = b;}
    public void run() {
        b.withdraw(700);}}
public class as_10_q_18 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        Bank18 b = new Bank18();
        new Customer18(b).start();
        new Customer18(b).start();
    }}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
Withdrawn 700
Insufficient
```

19. Write a program where multiple threads access two different objects but synchronize using a static synchronized method.

```
class Demo19 {
    static synchronized void show() {
        System.out.println(Thread.currentThread().getName());}}
class MyThread19 extends Thread {
    public void run() {
        Demo19.show();}}
class as_10_q_19 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        new MyThread19().start();
        new MyThread19().start();}}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3
Nishant - 241551078
Thread-0
Thread-1
```

20. Demonstrate the difference between object-level synchronization and class-level synchronization.

```
class Demo20 {
    synchronized void objLock() {
        System.out.println("Object lock");}
```

```java
    static synchronized void classLock() {
        System.out.println("Class lock");
    }}
public class as_10_q_20 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        Demo20 obj = new Demo20();
        obj.objLock();
        Demo20.classLock();
    }}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
Object lock
Class lock
```

21. Write a Java program that intentionally creates a deadlock using two threads and two shared resources.

```java
class A21 {}
class B21 {}
public class as_10_q_21 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        A21 a = new A21();
        B21 b = new B21();
        Thread t1 = new Thread(() -> {
            synchronized (a) {
                synchronized (b) {
                    System.out.println("Thread1");}}});
        Thread t2 = new Thread(() -> {
            synchronized (b) {
                synchronized (a) {
                    System.out.println("Thread2");
                }}});
        t1.start();t2.start();
    }}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
Thread1
Thread2
```

22. Modify the above program to avoid deadlock using proper resource ordering.

```java
class Resource22 {}
public class as_10_q_22 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        Resource22 r1 = new Resource22(), r2 = new Resource22();
        Thread t1 = new Thread(() -> {
            synchronized (r1) {
                synchronized (r2) {
                    System.out.println("Thread1 done");}}});
        Thread t2 = new Thread(() -> {
            synchronized (r1) {
                synchronized (r2) {
                    System.out.println("Thread2 done");}}});
        t1.start();t2.start();}}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
Thread1 done
Thread2 done
```

23. Implement a Producer–Consumer problem using wait() and notify().

```
class Buffer23 {
    int data;
    boolean available = false;
    synchronized void produce(int val) throws Exception {
        while (available) wait();
        data = val;
        available = true;
        System.out.println("Produced " + val);
        notify();}
    synchronized void consume() throws Exception {
        while (!available) wait();
        System.out.println("Consumed " + data);
        available = false;
        notify();
    }}
class as_10_q_23 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        Buffer23 b = new Buffer23();
        new Thread(() -> {
            try {
                b.produce(10);
            } catch (Exception e) {}}).start();
        new Thread(() -> {
            try {
                b.consume();
            } catch (Exception e) {}}).start();}}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3
Nishant - 241551078
Produced 10
Consumed 10
```

24. Write a program where one thread waits for another thread to deposit money before withdrawing.

```
class Account24 {
    int balance = 0;
    synchronized void withdraw(int amt) throws Exception {
        while (balance < amt) wait();
        balance -= amt;
        System.out.println("Withdraw done");}
    synchronized void deposit(int amt) {
        balance += amt;
        System.out.println("Deposited " + amt);
        notify();}}
class as_10_q_24 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        Account24 a = new Account24();
        new Thread(() -> {
            try {
                a.withdraw(500);
            } catch (Exception e) {}
        }).start();
        new Thread(() -> {
            a.deposit(1000); }).start();}}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\l
Nishant - 241551078
Deposited 1000
Withdraw done
```

25. Create a program demonstrating notifyAll() with multiple waiting threads.

```
class NotifyDemo25 {
    synchronized void wakeAll() {
        notifyAll();
        System.out.println("All threads notified");
    }}
public class as_10_q_25 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        new NotifyDemo25().wakeAll();
    }}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
All threads notified
```

26. Write a program demonstrating daemon thread behavior and show how JVM terminates it when user threads finish.

```
class DaemonDemo extends Thread {
    public void run() {
        while (true) System.out.println("Daemon running");
    }}
public class as_10_q_26 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        DaemonDemo t = new DaemonDemo();
        t.setDaemon(true); t.start();
        System.out.println("Main thread ends");
    }}
```

OUTPUT

```
Nishant - 241551078
Main thread ends
Daemon running
Daemon running
Daemon running
Daemon running
```

27. Write a program that interrupts a sleeping thread and handles the InterruptedException.

```
class SleepThread extends Thread {
    public void run() {
        try {
            System.out.println("Thread going to sleep");
            Thread.sleep(5000);
            System.out.println("Thread woke up");
        } catch (InterruptedException e) {
            System.out.println("Thread interrupted");
        }}}
class as_10_q_27 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        SleepThread t = new SleepThread();
        t.start();
        t.interrupt();
    }}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
Thread going to sleep
Thread interrupted
```

28. Implement a program that checks thread interruption status using isInterrupted().

```java
class CheckInterrupt extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            if (Thread.currentThread().isInterrupted()) {
                System.out.println("Thread interrupted");
                return;}
            System.out.println(i);
        }}}
public class as_10_q_28 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        CheckInterrupt t = new CheckInterrupt();
        t.start();
        t.interrupt();
    }}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
Thread interrupted
```

29. Implement a program demonstrating Reentrant Synchronization where one synchronized method calls another synchronized method.

```java
class ReentrantDemo {
    synchronized void method1() {
        System.out.println("Method1 start");
        method2();
        System.out.println("Method1 end");}
    synchronized void method2() {
        System.out.println("Method2 executing");}}
class as_10_q_29 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        new ReentrantDemo().method1();}}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
Method1 start
Method2 executing
Method1 end
```

30. Create a multi-threaded file downloader simulation where multiple threads download parts of a file concurrently.

```java
class Downloader extends Thread {
    int part;
    Downloader(int p) {
        part = p; }
    public void run() {
        System.out.println("Downloading part " + part); }}
public class as_10_q_30 {
    public static void main(String[] args) {
        System.out.println("Nishant - 241551078");
        for (int i = 1; i <= 5; i++) new Downloader(i).start();}}
```

OUTPUT

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.3\
Nishant - 241551078
Downloading part 5
Downloading part 4
Downloading part 3
Downloading part 2
Downloading part 1
```